```
-- ======================================================================
-- DailyBudget — Supabase database setup
-- Run this once in Supabase: Dashboard → SQL Editor → New query → Run.
-- Safe to re-run: tables use IF NOT EXISTS and policies are recreated.
-- ======================================================================

create extension if not exists pgcrypto;

-- ---------- Helper: keep updated_at current ----------
create or replace function public.set_updated_at()
returns trigger language plpgsql set search_path = '' as $$
begin
  new.updated_at := now();
  return new;
end $$;

-- ---------- Helper: stop records being moved to another owner ----------
create or replace function public.prevent_owner_change()
returns trigger language plpgsql set search_path = '' as $$
begin
  if new.user_id is distinct from old.user_id then
    raise exception 'Changing the owner of a record is not allowed';
  end if;
  return new;
end $$;

-- ---------- Tables ----------
create table if not exists public.profiles (
  id            uuid primary key references auth.users(id) on delete cascade,
  display_name  text not null default '' check (char_length(display_name) <= 80),
  created_at    timestamptz not null default now(),
  updated_at    timestamptz not null default now()
);

create table if not exists public.preferences (
  user_id          uuid primary key references auth.users(id) on delete cascade,
  currency         text not null default 'USD' check (currency ~ '^[A-Z]{3}$'),
  daily_limit      numeric(14,2) not null default 0 check (daily_limit >= 0 and d
  rollover         boolean not null default false,
  opening_balance  numeric(14,2) not null default 0 check (abs(opening_balance) <
  theme            text not null default 'system' check (theme in ('system','ligh
  created_at       timestamptz not null default now(),
  updated_at       timestamptz not null default now()
);
```

```
create table if not exists public.bills (
  id          uuid primary key default gen_random_uuid(),
  user_id     uuid not null default auth.uid() references auth.users(id) on delet
  name        text not null check (char_length(name) between 1 and 80),
  category    text not null check (category in ('rent','electricity','water','int
  amount      numeric(14,2) not null check (amount > 0 and amount < 1000000000),
  due_date    date not null check (due_date between '1970-01-01' and '2100-12-31'
  recurrence  text not null default 'monthly' check (recurrence in ('once','weekl
  status      text not null default 'unpaid' check (status in ('unpaid','paid')),
  notes       text not null default '' check (char_length(notes) <= 300),
  is_sample   boolean not null default false,
  created_at  timestamptz not null default now(),
  updated_at  timestamptz not null default now(),
  unique (id, user_id)   -- lets transactions reference a bill only within the sa
);

create table if not exists public.transactions (
  id              uuid primary key default gen_random_uuid(),
  user_id         uuid not null default auth.uid() references auth.users(id) on d
  type            text not null check (type in ('income','expense','transfer')),
  amount          numeric(14,2) not null check (amount > 0 and amount < 100000000
  currency        text not null default 'USD' check (currency ~ '^[A-Z]{3}$'),
  txn_date        date not null check (txn_date between '1970-01-01' and '2100-12
  category        text not null check (char_length(category) between 1 and 60),
  description     text not null default '' check (char_length(description) <= 200
  payment_method  text not null default 'cash' check (payment_method in ('cash','
  essential       boolean not null default true,
  notes           text not null default '' check (char_length(notes) <= 1000),
  bill_id         uuid,
  is_sample       boolean not null default false,
  created_at      timestamptz not null default now(),
  updated_at      timestamptz not null default now(),
  -- A payment can only link to a bill owned by the same user. Deleting the bill
  constraint transactions_bill_fk foreign key (bill_id, user_id)
    references public.bills (id, user_id) on delete set null (bill_id)
);

create table if not exists public.budgets (
  id          uuid primary key default gen_random_uuid(),
  user_id     uuid not null default auth.uid() references auth.users(id) on delet
  month       date not null check (extract(day from month) = 1),
  category    text not null check (char_length(category) between 1 and 60),
  amount      numeric(14,2) not null check (amount > 0 and amount < 1000000000),
  is_sample   boolean not null default false,
  created_at  timestamptz not null default now(),
  updated_at  timestamptz not null default now(),
```

```
  unique (user_id, month, category)
);

create table if not exists public.savings_goals (
  id             uuid primary key default gen_random_uuid(),
  user_id        uuid not null default auth.uid() references auth.users(id) on de
  name           text not null check (char_length(name) between 1 and 80),
  kind           text not null default 'general' check (kind in ('emergency','hou
  target_amount  numeric(14,2) not null check (target_amount > 0 and target_amoun
  saved_amount   numeric(14,2) not null default 0 check (saved_amount >= 0 and sa
  target_date    date check (target_date between '1970-01-01' and '2100-12-31'),
  description    text not null default '' check (char_length(description) <= 300)
  is_sample      boolean not null default false,
  created_at     timestamptz not null default now(),
  updated_at     timestamptz not null default now()
);

-- ---------- Indexes ----------
create index if not exists transactions_user_date_idx on public.transactions (use
create index if not exists transactions_user_cat_idx  on public.transactions (use
create index if not exists transactions_bill_idx      on public.transactions (bil
create index if not exists budgets_user_month_idx     on public.budgets (user_id,
create index if not exists bills_user_due_idx         on public.bills (user_id, d
create index if not exists goals_user_idx             on public.savings_goals (us

-- ---------- Triggers ----------
do $$
declare t text;
begin
  foreach t in array array['profiles','preferences','transactions','budgets','bil
    execute format('drop trigger if exists %I_updated_at on public.%I', t, t);
    execute format('create trigger %I_updated_at before update on public.%I for e
  end loop;
  foreach t in array array['preferences','transactions','budgets','bills','saving
    execute format('drop trigger if exists %I_owner_lock on public.%I', t, t);
    execute format('create trigger %I_owner_lock before update on public.%I for e
  end loop;
end $$;

-- ---------- Create profile + preferences for each new user ----------
create or replace function public.handle_new_user()
returns trigger language plpgsql security definer set search_path = '' as $$
begin
  insert into public.profiles (id, display_name)
  values (new.id, left(coalesce(new.raw_user_meta_data->>'display_name', ''), 80)
  on conflict (id) do nothing;
  insert into public.preferences (user_id) values (new.id) on conflict (user_id)
```

```
  return new;
end $$;

drop trigger if exists on_auth_user_created on auth.users;
create trigger on_auth_user_created after insert on auth.users
  for each row execute function public.handle_new_user();

-- ---------- Row Level Security ----------
alter table public.profiles      enable row level security;
alter table public.preferences   enable row level security;
alter table public.transactions  enable row level security;
alter table public.budgets       enable row level security;
alter table public.bills         enable row level security;
alter table public.savings_goals enable row level security;

-- profiles (owner column is "id")
drop policy if exists "profiles_select_own" on public.profiles;
drop policy if exists "profiles_insert_own" on public.profiles;
drop policy if exists "profiles_update_own" on public.profiles;
drop policy if exists "profiles_delete_own" on public.profiles;
create policy "profiles_select_own" on public.profiles for select to authenticate
create policy "profiles_insert_own" on public.profiles for insert to authenticate
create policy "profiles_update_own" on public.profiles for update to authenticate
create policy "profiles_delete_own" on public.profiles for delete to authenticate

-- all other tables (owner column is "user_id")
do $$
declare t text;
begin
  foreach t in array array['preferences','transactions','budgets','bills','saving
    execute format('drop policy if exists %I on public.%I', t||'_select_own', t);
    execute format('drop policy if exists %I on public.%I', t||'_insert_own', t);
    execute format('drop policy if exists %I on public.%I', t||'_update_own', t);
    execute format('drop policy if exists %I on public.%I', t||'_delete_own', t);
    execute format('create policy %I on public.%I for select to authenticated usi
    execute format('create policy %I on public.%I for insert to authenticated wit
    execute format('create policy %I on public.%I for update to authenticated usi
    execute format('create policy %I on public.%I for delete to authenticated usi
  end loop;
end $$;

-- ---------- Table privileges: nothing for anonymous visitors ----------
revoke all on public.profiles, public.preferences, public.transactions, public.bu
grant select, insert, update, delete on public.profiles, public.preferences, publ

-- ---------- Self-service account deletion ----------
-- Deletes the calling user from auth.users; ON DELETE CASCADE removes all their
```

```
create or replace function public.delete_my_account()
returns void language plpgsql security definer set search_path = '' as $$
declare uid uuid := auth.uid();
begin
  if uid is null then
    raise exception 'Not authenticated';
  end if;
  delete from auth.users where id = uid;
end $$;

revoke all on function public.delete_my_account() from public, anon;
grant execute on function public.delete_my_account() to authenticated;
```